

Scalable DNS Configuration Verification via the LEC-native Query Resolution Graph

ABSTRACT

Existing DNS configuration verifiers struggle to scale and provide limited support for frequent incremental updates. Motivated by the resemblance between authoritative DNS resolution and data-plane forwarding, we present AETHER, which models authoritative DNS resolution as a *query resolution graph* for efficient and incremental verification. Unlike prior systems that build *global*, record-name-based equivalence classes that remain fine-grained and expensive, AETHER is *LEC-native*: it constructs local equivalence classes (LECs) by grouping domain names with identical match-action behavior at each nameserver, and verifies directly on a compact graph. AETHER consists of (1) a domain-name-space encoding for efficient LEC manipulation, (2) a symbolic executor that explores all resolution paths without enumerating names, and (3) an incremental mechanism that localizes recomputation to changed graph regions. On real-world datasets with over 6 million resource records, AETHER achieves up to 255.7 $\times$ speedup for end-to-end verification and up to 3,370 $\times$ for incremental verification.

1 INTRODUCTION

Over the past three decades, since the initial publication of two RFCs [34, 35] defining the basic Domain Name System (DNS), the DNS system has evolved into a crucial component of the Internet. As one of the largest distributed hierarchical database systems, its immense scale and complexity pose significant challenges for configuration [3, 35, 36, 39, 43, 44]. Moreover, its configurations are updated extremely frequently and commonly. Any failure in the DNS could lead to incalculable losses [8, 13, 14, 20, 45, 49].

Static methods cannot detect complex errors. To ensure uninterrupted DNS operations, operators employ various techniques, primarily centered on probing (e.g., linting and testing) [2, 12, 18, 33, 38, 57] and the static analysis of individual files [9, 21, 52]. However, these tools have limited support for verifiable properties and often lack precision. For example, linting-based approaches primarily detect basic syntax and formatting issues, but fail to capture complex logical errors or context-dependent misconfigurations.

Existing DNS configuration verification tools are inefficient. Some studies [10, 23, 29, 48] aim to address the limitations of static DNS analysis. Among them, GRoot [23] and

the subsequent work [29] provide the most complete, state-of-the-art verification: they formalize authoritative DNS resolution semantics and use a centralized architecture to collect zone files and perform cross-nameserver behavioral analysis. However, both still suffer from performance limitations.

First, GRoot demonstrates inefficiency on large dataset. For example, it takes GRoot 10⁴s (~3h) to process 10 million resource records [23]. It could limit its practicality in scenarios that require timely verification. Second, GRoot is designed for global verification of the entire DNS configuration. When operators perform incremental updates, the algorithm's structure does not support verifying only the affected portion of the configuration; instead, it requires reprocessing the entire namespace. As a result, GRoot does not support incremental verification.

Key Insights. In this paper, we address the challenge of building DNS configuration verification tools that are efficient, complete, and incremental. Our key insight is that *authoritative DNS resolution forms a match-action pipeline*: queries are matched against resource records and trigger actions such as answering, delegating, or rewriting (e.g., CNAME/DNAME). Chained across nameservers, this behavior resembles packet forwarding, suggesting that data-plane verification (DPV) techniques [1, 17, 22, 26, 27, 31, 32, 41, 46, 47, 50, 51, 53–55, 58, 59] can be adapted to detect DNS misconfigurations.

A core DPV acceleration principle is *local aggregation*: inputs inducing the same action are merged into local equivalence classes (LECs) so verification operates on aggregated spaces. DNS offers the same opportunity because many record-defined names trigger identical actions (e.g., delegation to the same nameserver set). However, GRoot reasons at per-record domain-name granularity: its global equivalence classes (GECs) are built from names appearing in zonefiles and are effectively represented by exact domain names. As a result, action-identical name spaces are not merged, producing excessive ECs and repeated traversal of raw match rules, which limits scalability, especially with rewrites.

These observations motivate an *LEC-native* verifier: it aggregates the domain-name query space locally and reasons directly over the resulting query resolution graph, rather than refining queries into global ECs. This graph-centric design also naturally enables incremental verification, since updates can be localized to the affected nodes and LECs without recomputing global partitions. The main challenge is that

Error	Description
Delegation Inconsistency	For a trace that the parent node with the answer type is Ref and the child with the answer type is ANS but do not have the same set of NS and A records for delegation.
Lame Delegation	For a trace that the answer type returns REFUSED.
Missing Glue Records	For a trace that the answer type in any log is NS but not followed by the A (glue) records.
Non-Existent Domain	For a trace that the answer returns NXDOMAIN.
Cyclic Zone Dependency	For a trace that there exists two logs that are the same.
Rewriting Loop	For a trace that there exists a rewrite that result in a loop.
Query Exceeds Maximum Length	For a trace that the length of the input query or the number of the labels exceeds the maximum number.
Rewrite Blackholing	For a trace that after the rewrite log the next log returns NXDOMAIN.

Table 1: Types of DNS misconfiguration that our framework support. We derive these categories from GRoot [23], a subsequent study [29] and RFC 8484 [16].

the DNS query space fundamentally differs from the DPV header space. Packet headers are fixed-length bit-vectors that admit standard encodings (e.g., tries or BDDs), whereas domain names are *variable-length label sequences* with wildcard and suffix-based matching semantics.

To overcome these limitations, we design AETHER, a generic DNS configuration verification framework that adopts an LEC-native query resolution graph for scalable and incremental verification, with the following key designs:

D1: A new data structure for domain-name-space encoding and a compact query resolution graph (§3). To efficiently manipulate LECs, we design a configuration-aware encoding that uses Boolean variables only for the label values and name depths present in the DNS configuration, and represent the resulting variables with BDDs. By aggregating query spaces locally with BDDs, LECs compact the query resolution graph; reasoning over LECs rather than per-record global ECs further shrinks the exploration space and significantly improves verification scalability.

D2: A symbolic query execution algorithm for full query-space coverage (§4). Starting from the entrance nameserver and a BDD-represented symbolic query, AETHER intersects the query space with LEC-labeled edges and propagates the resulting subspaces according to actions (delegate, rewrite, or terminate), generating traces without enumerating domain names. A path-count bound and a visited-state cache ensure termination and prune rewrite-induced cycles, while a lightweight prefix filter reduces the number of costly BDD intersections.

D3: An incremental verification algorithm for configuration changes (§5). Upon updates, AETHER regenerates only the affected LECs and identifies the corresponding changed nodes in the query resolution graph. It then replays symbolic execution from these nodes and verifies only the impacted resolution paths. By localizing recomputation to the affected portion of the graph, AETHER enables efficient incremental verification without reprocessing the entire domain-name space.

Evaluation. We implement a prototype of AETHER and evaluate it with a real-world open-source dataset [7] and a university dataset. We compare AETHER with GROOT, the

first DNS configuration verification tool. We use the university dataset to evaluate verification capabilities. On the open-source dataset, we measure end-to-end verification time, including EC/LEC construction, symbolic execution, and incremental-update processing. For end-to-end verification, AETHER outperforms GROOT by up to 255.7×. Under incremental updates, AETHER achieves up to 3,370× speedup.

2 OVERVIEW

This section introduces some key concepts of DNS, and illustrates key designs of AETHER using an example.

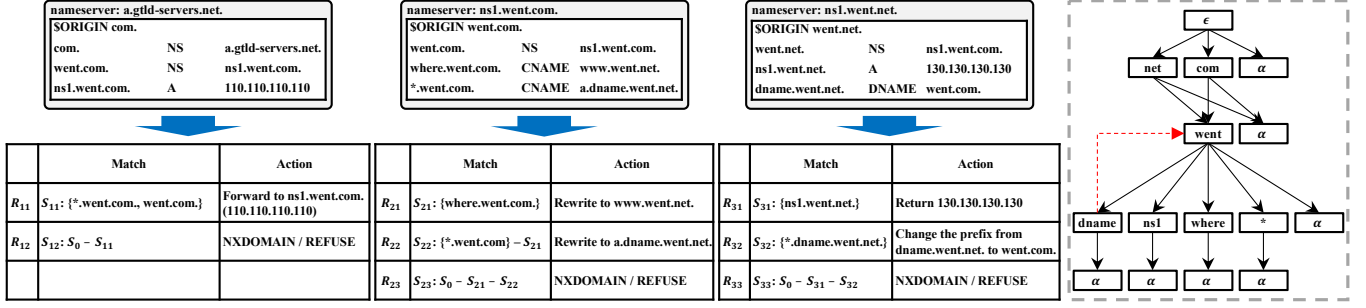
2.1 Basic Concept

A Simplified Model of DNS. Generally, the DNS system comprises three major components [28]: (i) clients, which initiate queries and receive replies; (ii) resolvers, which issue queries to nameservers and return answers to clients; and (iii) nameservers, which store DNS records and respond to queries. In practice, The resolver can be further subdivided into recursive resolvers and iterative resolvers.

We study DNS configuration verification, and thus focus on nameservers while abstracting away clients and resolvers. Concretely, we model redirections as occurring directly among nameservers, and omit the resolver-nameserver interaction details. We also exclude mechanisms that primarily affect resolver-side behavior, including DNS caching and DNS Security Extensions (DNSSEC) [37], to keep the model focused and tractable. This modeling choice is consistent with prior work such as GROOT [23].

A nameserver is configured by one or more configuration files, commonly referred to as zonefiles. Each zonefile contains a set of resource records (RRs) that map domain names to the data needed for DNS resolution. An RR has five fields: name, TTL, class, type, and data. The name identifies the domain to which the record applies (e.g., *www.google.com*). The TTL (Time To Live) specifies how long the record may be cached. The class indicates the network class (typically IN for the Internet). The type specifies the kind of mapping the record encodes, e.g., NS designates the authoritative nameserver for a domain, A and AAAA provide IPv4 and IPv6 addresses, CNAME aliases one domain name to another, and

(a) Zonefiles of three nameservers where the first RR is the self identification rule.

(b) LEC results of Aether where S_0 denotes the full query space.

(c) EC results of GROOT.

Figure 1: A three-nameserver DNS example illustrating domain-name-space partition: AETHER computes local equivalence classes (LECs) per nameserver, whereas GROOT constructs global ECs across nameservers.

DNAME rewrites a domain and all of its subdomains to a different domain. The data field carries the corresponding payload (e.g., an IP address or another domain name).

Since the class field is almost always IN in practice, we omit it in the remainder. While TTL is important for resolver caching, our focus is on nameserver-side configuration semantics, so we also exclude TTL from consideration. Accordingly, we represent an RR as a triple $\langle rname, rtype, rdata \rangle$, where $rname$ is the domain name, $rtype$ is the record type, and $rdata$ is the record data.

Match-Action View of Nameserver Resolution. For a given nameserver, we abstract its resolution behavior as a match-action table consisting of a set of resolution rules. Each rule includes (i) a matching condition over domain names and (ii) an action to take upon a match. Actions fall into two classes. A terminate action returns an answer (e.g., the resolved data) or signals a failure. A forward action delegates resolution to another nameserver, which may be the same nameserver (via an aliasing step) or a different authoritative server (via referral). When a query $q = \langle qname, qtype \rangle$ arrives, the nameserver evaluates rules according to its defined priority order and applies the action of the first applicable match.

Query Log and Trace. We introduce the notion of a *query trace* to capture the end-to-end processing of a DNS query, and use it to define query behavior in the DNS system. As a query q traverses the system, each successful match against a resolution rule produces a *query log*. A log records (i) the query's original value, (ii) its resulting value after the rule is applied (if the name is rewritten), and (iii) the location (nameserver and zone context) where the match occurs.

A *query trace* for q is an ordered sequence of query logs whose final log must execute a *terminate* action. In contrast, *forward* actions may introduce branching because the next-hop nameservers are not always uniquely determined. For example, in some deployments queries may be issued to multiple candidates. Consequently, a single query q can induce

multiple feasible traces. Query traces form the basis of our verification semantics: for each property, we validate it over the set of all feasible traces permitted by the configuration.

2.2 Key Designs of AETHER

Similar to GROOT [23], we assume a centralized verifier that collects and analyzes all relevant configuration files. We focus on common DNS misconfigurations, such as cases where a rewritten query never yields an answer (i.e., *rewrite blackholing*) or where rewriting induces cyclic resolution (i.e., *rewrite loops*). Table 1 summarizes all properties supported by AETHER.

We illustrate AETHER's workflow using the example in Figure 1 with an input query $q_{\text{origin}} : \langle *.com., A \rangle$. In this running example, we verify three properties: (i) no rewrite blackholing, (ii) no rewrite loops, and (iii) eventual termination. We further demonstrate incremental verification after fixing the rewrite-blackholing misconfiguration. For clarity, we consider only A queries and omit the query type in the remainder; thus, we denote the input as $q_{\text{origin}} : \{.com.\}$. The workflow proceeds as follows.

2.2.1 Query Resolution Graph. An authoritative nameserver resolves a query by matching its queried name against the RRs in its zonefiles and applying the corresponding actions. We therefore model a nameserver as a *match-action* component with one logical *input port* and multiple logical *output ports*. Each output port corresponds to an action (e.g., terminate with an answer, terminate with an error, forward via delegation, rewrite via aliasing), and is associated with the set of domain names that trigger that action.

To compute these action-specific name sets, AETHER first orders RRs by specificity, i.e., a more specific domain name has higher priority than a less specific one. Each RR is then translated into a domain-name subspace. Subspaces that lead to the same action are merged into a *local equivalence class (LEC)*, such that all names in an LEC exhibit identical resolution behavior at this nameserver (§4).

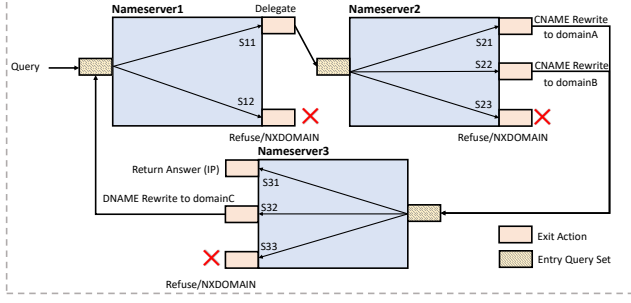


Figure 2: The DNS query resolution graph model corresponding to the example in Figure 1. Here domainA represents *www.went.net.*, domainB represents *a.dname.went.net.* and domainC represents *went.com.*

We represent the input and output ports as nodes in a *query resolution graph*. The nameserver’s input node connects to each of its output nodes, and the edge between them is annotated with the LEC that triggers the action. Next, for each *forward* output node, AETHER links it to the input node of the next-hop nameserver(s) implied by the delegation action. Repeating this process across all nameservers yields a global query resolution graph for verification.

Example. Figure 1 shows how AETHER computes LECs for each nameserver for the running example. We merge the NXDOMAIN and REFUSE behaviors, since both correspond to a *no-answer* outcome for the query. Let S_0 denote the universe of query names under consideration. For nameserver *a.gtld-servers.net*, AETHER partitions S_0 into action-consistent subspaces, including:

1. $S_{11} = \{\text{went.com.}, *.went.com.\}$, whose queries are forwarded to nameserver *ns1.went.com.*;
2. $S_{12} = S_0 - S_{11}$, whose queries do not reach the intended delegation. S_{12} can be further refined by termination outcome, e.g., $\{\text{com.}, *.com.\} - S_{11}$ for NXDOMAIN and $S_0 - \{\text{com.}, *.com.\}$ for REFUSED.

Similarly, we compute the partitions for the other nameservers. For *ns.went.com*, the match-action table yields two forwarding outputs: a CNAME rewrite to *www.went.net* and a CNAME rewrite to *a.dname.went.net*. Both rewrites delegate resolution to the same next-hop nameserver, *ns1.went.net*. By connecting each forwarding output to the corresponding next-hop input, AETHER links nameservers into a single graph model. Figure 2 shows the resulting end-to-end query resolution graph.

2.2.2 Trace Generation and Property Check. Once the query resolution graph is constructed, the process of generating query traces can begin. This process can be implemented using a depth-first search (DFS) algorithm. Given a query q_{in} and the starting domain nameserver, AETHER starts from the input port of the nameserver, and traversals the query resolution graph. If the action is of a termination type, we directly obtain a log. Otherwise, we execute the behavior

specified by the action, obtain the query space q_{out} to be forwarded (which may be the same as q_{in} or different), record the information to generate a log, and forward q_{out} as a new request to the next nameserver. This process continues until the final action is of the terminate type, at which point a complete trace is generated. It is worth noting that due to the forwarding operation, q_{out} may be forwarded to multiple domain nameservers. As a result, the logs of different traces may overlap. However, this does not affect the definition or validity of individual traces.

Consider the scenario in Figure 3a, where a query q_{origin} starts at the domain nameserver *a.gtld-server.net*. The query matches rule R_{11} , generating a log L_1 , where $q_{in1} = q_{out1} = S_{11}$. The query q_{out1} is then forwarded to the *ns1.went.com* nameserver.

At *ns1.went.com*, we focus on the partial subspaces where the query matches successfully. The first match is with rule R_{21} , which modifies the input query $q_{in2} = \{\text{where.went.com.}\}$ to a new output query $q_{out2} = \{\text{www.went.net.}\}$ and forwards it to *ns1.went.net*. A log L_2 is generated for this match. Subsequently, at *ns1.went.net*, the query q_{out2} matches rule R_{33} , generating a log L_4 . Here, $q_{in3} = q_{out3} = \{\text{www.went.net.}\}$. Since the action corresponding to R_{33} is of the terminate type, the first trace is formed as: $T_1 = [L_1, L_2, L_4]$.

Next, we consider another match for q_{out1} at *ns1.went.com.*, this time with rule R_{22} . The action for R_{22} modifies the query $q_{in3} = S_{22}$ to $q_{out3} = \{\text{a.dname.went.net.}\}$ and forwards it to *ns1.went.net.*, generating a log L_3 . At *ns1.went.net.*, the query q_{out3} matches rule R_{33} , which modifies $q_{in5} = \{\text{a.dname.went.net.}\}$ to $q_{out5} = \{\text{a.went.com.}\}$, generating a log L_5 . The query is then forwarded back to *ns1.went.com.*, where it matches R_{23} again, generating a log L_6 . Although further forwarding might occur, we stop here because of loop detection and define the second trace as: $T_2 = [L_1, L_4, L_5, L_6]$.

After obtaining these two traces, we can proceed to verify properties. For a given trace, AETHER examines whether its behavior aligns with the semantics of specified properties. Analyzing trace T_1 , we find that a rewrite operation occurs in L_2 , and the final action is NXDOMAIN. Therefore, T_1 violates the no-rewrite-blackholing property.

Similarly, analyzing trace T_2 , we observe that rewrite operations occur in both L_3 and L_5 . Furthermore, the query is forwarded to *ns1.went.com* in both cases, and the intersection of q_{in3} and q_{in6} is $\{\text{a.went.com}\}$, indicating that *ns1.went.com* received the same query twice. Therefore, T_2 violates the no-rewriting-loop property.

2.2.3 Incremental Verification. AETHER supports *incremental verification* under configuration updates. The key challenge is to update verification results quickly without redoing work that is unaffected by the change. A straightforward solution would rebuild the entire query resolution

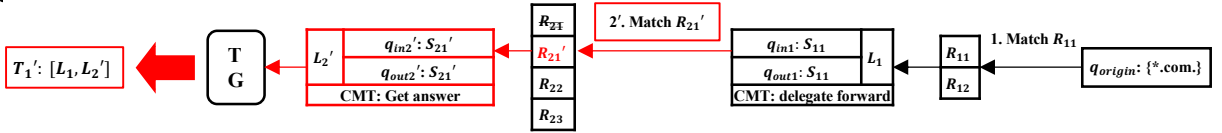
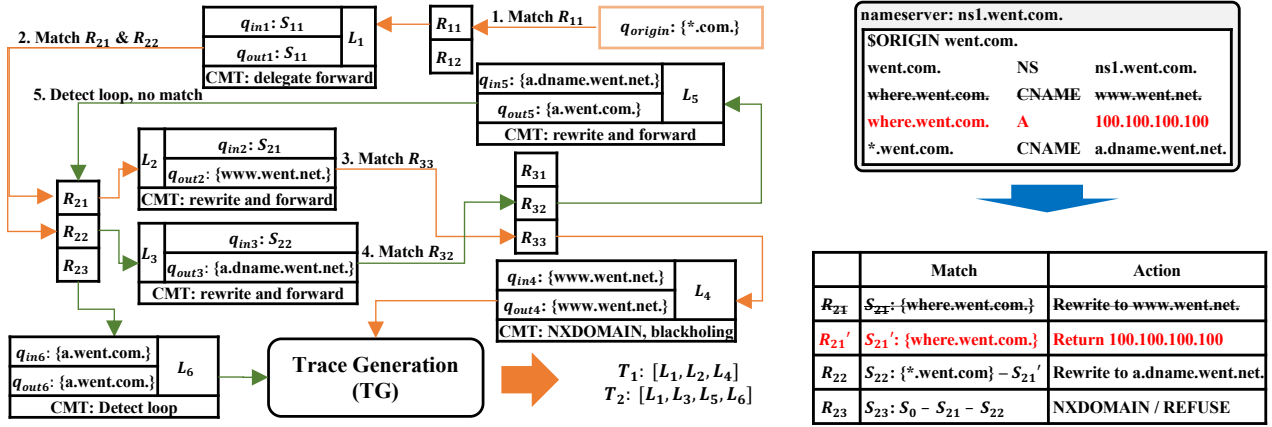


Figure 3: The symbolic execution and property-checking workflow of AETHER, including incremental execution and verification, for the example DNS in Figure 1. CMT denotes a brief comment in a log.

graph and rerun symbolic execution, but this is inefficient in practice: most zonefiles remain unchanged, and the vast majority of traces are unaffected.

AETHER instead reuses the existing query resolution graph and localizes recomputation to the changed region. Specifically, upon a configuration update, AETHER (i) recomputes the query-space partition (i.e., LECs) only for the nameserver(s) whose zonefiles changed, (ii) identifies the affected output ports (graph nodes/edges) whose match-action behaviors differ, and (iii) replays symbolic execution *only* from these changed nodes. Using the cached symbolic state (query space and partial traces) up to the change point, AETHER generates the new suffix traces and re-checks the properties on the impacted traces. This design avoids redundant computation incurred by full recomputation.

Example. Consider Figure 3b. Suppose the record $R_{21} = \langle where.went.com., CNAME, www.went.net. \rangle$ is replaced by an A record $R_{21}' = \langle where.went.com., A, 100.100.100.100 \rangle$. AETHER recomputes the match-action table (and the corresponding LECs) only for the *ns1.went.com.* nameserver, and detects that the node n_1 induced by R_{21} is replaced by node n_2 induced by R_{21}' .

AETHER then regenerates all traces starting from n_2 for the affected query subspace at the *ns1.went.com.* nameserver. This yields an updated log L_2' that terminates with the IPv4 address 100.100.100.100. Accordingly, trace T_1 is updated to $T_1' = [L_1, L_2']$, which now satisfies the no-rewrite-blackholing property.

Comparison with GROOT. As discussed in Section 1, AETHER adopts Local Equivalence Classes (LECs) instead of record-level Global Equivalence Classes (GECs) in GROOT for verification, performing queries based on domain-name spaces rather than individual domain names. This shift significantly reduces the number of equivalence classes (ECs), which in turn lowers both the generation overhead and the number of verification rounds. Additionally, the match overhead at each nameserver is reduced due to fewer match conditions/items.

Figure 1c illustrates the label graph generated by GROOT, which results in 22 global ECs. Since the label graph is created for each query type, the total number of global ECs is $22T$, where T represents the number of query types (such as A and CNAME). During verification, all $22T$ ECs must be processed through the match-action tables of nameservers to generate query traces and check properties. Each EC in GROOT corresponds to a specific domain name, meaning the match-action table is essentially a raw table containing all resource records, leading to high match overhead. In contrast, as shown in Figure 2, AETHER can merge resource records with identical resolution behaviors, resulting in a more compact query resolution graph with reduced overhead.

3 QUERY RESOLUTION GRAPH

To enable scalable reasoning over DNS behaviors, we partition all possible DNS queries, denoted by $\{qname, qtype\}$, at each nameserver into a set of Local Equivalence Classes

(LECs). An LEC captures DNS queries that exhibit identical local resolution behavior (e.g., answer, rewrite, or refuse) under a nameserver's zone files. LECs abstract the zone files of each nameserver into a match-action table, whose actions induce the transitions in the query resolution graph.

We compute LECs at the nameserver level because zone-files on the same nameserver may cover overlapping or nested domain spaces (e.g., *a.com* and *b.a.com*). Generating LECs per zonefile would therefore result in overlapping classes. Computing LECs jointly across all zonefiles ensures that the resulting LECs form a mutually exclusive partition of the query space. Such overlap does not occur across different nameservers.

3.1 LEC Encoding

We use Binary Decision Diagrams (BDDs) to encode LECs. BDDs have been widely used in data-plane verification systems [26] to encode packet header spaces. However, these techniques cannot be directly applied to DNS verification for two reasons. First, existing data-plane verification tools (e.g., AP [53, 54], Veriflow [27]) do not support query rewrites, which are fundamental to DNS resolution. Second, unlike packet headers (e.g., IP) with fixed-length fields, DNS domain names are variable-length. A domain name consists of multiple labels, each of which can be up to 63 characters long, with a maximum total length of 255 characters. Directly encoding domain names at the character level would therefore require up to $255 \times 8 = 2040$ Boolean variables (8 bits per character), making BDD operations prohibitively expensive.

Compact domain name encoding. To minimize the number of variables required for domain name encoding, we focus the encoding on domain names that can occur at the nameservers under verification, rather than arbitrary domain names. This substantially reduces the size of the domain name space and enables a compact Boolean representation.

Specifically, we scan the zone files of all nameservers to determine (1) the total number of distinct labels appearing in the *rname* field of all resource records and in the *rdata* field for CNAME and DNAME records, denoted as m ; and (2) the maximum number of labels in any domain name (i.e., domain name bound), denoted as n . Under this encoding, any domain name appearing in the zone files can be represented using $n \cdot \lceil \log_2 m \rceil$ Boolean variables.

Encoding the query type is straightforward. As there are only a fixed number of query types (i.e., A, AAAA, CNAME, MX, NS, etc.), each query type can be encoded using $\lceil \log_2 t \rceil$ Boolean variables, where t is the number of query types.

Handling DNAME Rewrites. A DNAME record rewrites a suffix of the queried domain name, potentially introducing additional labels and causing the maximum number of domain name labels to exceed the original bound n . To address the potential growth introduced by rewrites, we extend the

AType	AData	Description
ANSWER	IP	Return the certain IP.
DELEGATE	DN	Delegate the query to another server.
REWRITEC	DN	Rewrite the query name via CNAME.
REWRITED	DN	Rewrite the query name via DNAME.
REFUSE	-	Refuse to process the query.
NONEXIST	-	Return a notification that the domain does not exist.

Table 2: The DNS resolution behaviors. DN represents domain name.

label bound to $n + \delta$, where δ captures the maximum number of hops to detect rewrite loops (i.e., the maximum number of additional labels introduced during resolution). The value of δ can also be specified by the user to ensure sufficient label capacity under DNS rewrites.

Optimizations for DNS Hierarchy. In practice, the distribution of labels is uneven across different levels of the DNS hierarchy. As a result, allocating the same number of BDD variables to every label position is inefficient.

We therefore refine the encoding by allowing different hierarchy levels to use different label encodings. Labels closer to the root typically come from a smaller set and can be encoded using fewer variables, whereas labels at deeper levels exhibit greater diversity and require more variables. For example, if there are 4 distinct labels at the first level and 8 at the second level, we allocate 2 variables for the first level and 3 variables for the second level, instead of using a uniform allocation across all levels.

3.2 Match-action Table Computation

Using the LEC abstraction, we model each nameserver's zone files as a collection of match-action rules. In a rule $\langle qspace, action \rangle$, the match field *qspace* is a predicate over the query space (i.e., *qname* and *qtype*) corresponding to an LEC, and the action field *action* represents the common resolution behavior of queries in that LEC. These rules collectively form a match-action table that maps query spaces to resolution actions.

Given a nameserver s with zone file z , the action taken on a DNS query q is defined as $Action_{s,z}(q) = \langle atype, adata \rangle$ where *atype* specifies the type of resolution behavior and *adata* contains the data associated with that behavior. We formalize the action of nameservers into 6 types. Each action type and its resolution behavior are defined in Table 2.

These actions fall into two categories: 1) Termination, which concludes resolution at the current nameserver, including ANSWER, REFUSE, NONEXIST; 2) Forwarding, which propagates the query to subsequent resolution steps, including DELEGATE, REWRITEC, and REWRITED.

Algorithm 1: Computing the LECs for a nameserver

```

1 Function ConstructLECs(zonefiles):
2   sorted_zonefiles[]  $\leftarrow$  zonefiles sorted by decreasing
   number of labels in their origin domains;
3   LECtable(qspace, action)  $\leftarrow$  {};
4   remain_bdd  $\leftarrow$  S;
5   foreach zonefile (origin, records) in sorted_zonefiles do
6     zone_space  $\leftarrow$  GetSpace(origin, all_types, 1);
7     zone_match  $\leftarrow$  zone_space  $\cap$  remain_bdd;
8     remain_bdd  $\leftarrow$  remain_bdd  $-$  zone_match;
9     LECtable  $\leftarrow$  LECtable  $\cup$  ZoneLECs(records, zone_match)
10  refuse_action  $\leftarrow$  (Refuse, -);
11  LECtable  $\leftarrow$  LECtable  $\cup$  (remain_bdd, refuse_action);
12  return LECtable

```

Two-stage Match-action Table Computation. To ensure completeness and mutual exclusivity, we propose a two-stage approach to construct LECs, reflecting the hierarchical nature of DNS query processing. This design stems from the observation that DNS resolution actions are determined at different administrative levels:

- **Stage I (Nameserver-level):** The nameserver first determines whether a query should be refused by checking whether the query domain name matches the *origin* of any zone. Queries that do not match the origin of any configured zone are immediately refused. Accordingly, we first compute the LECs of REFUSE type at the nameserver level (Algorithm 1).
- **Stage II (Zonefile-level):** If the query matches the *origin* of a zone, the nameserver selects the corresponding zonefile for further processing (e.g., ANSWER, DELEGATE). We then independently construct zonefile-level LECs for each zonefile based on its resource records (Algorithm 2).

Query Space Representation Function. Both Stage I and II utilize the GetSpace function with three input parameters (*rname*, *types*, *flag*) to get the BDD-based representation of zones or records (i.e., *zone_space* in Algorithm 1 and *record_space* in Algorithm 2). The first two parameters *rname* and *types* specify the target domain name string and query types (Table 2). Notably, we use *all_types* to denote the universal set of action types. The third parameter *flag* is used to dictate three types of scope of domain. For example, given domain name *a.com*: 1) *flag* = 0 represents the domain itself (i.e., *a.com*.); 2) *flag* = 1 covers both the domain and its subdomains (i.e., *a.com*. and **.a.com*.); and 3) *flag* = 2 covers only the subdomains (i.e., **.a.com*.).

Notably, due to the hierarchical nature of DNS, the responsible domain spaces corresponding to records or zonefile origins may be nested within one another. For instance, *a.com* origin domain encompasses the *b.a.com* (a delegated subdomain). To ensure the computed LECs are mutually exclusive,

we use a match field to represent the exclusive space of a zone or record (e.g., *zone_match* and *record_match*), thus isolating the domain space of zones or records from any overlapping parent or child domains.

Nameserver-level Match-action Table Computation.

We perform two main tasks to construct the nameserver-level match-action table, including: 1) determine the exclusive space associated with each zone using a longest-prefix-matching strategy, and 2) compute the REFUSE LEC.

As shown in Algorithm 1, AETHER first sorts all zonefiles by the number of labels in their origin domains in descending order (line 2), so that more specific zones (e.g., *sub.example.com*) are processed before their parent zones (e.g., *example.com*), enforcing longest-prefix-matching semantics.

During the construction process, AETHER maintains a variable *remain_bdd*, initialized to the universal query space *S*, to track the portion of the query space not yet claimed by any zone. For each zonefile in order, AETHER computes its exclusive matching space *zone_match* by intersecting the zone's responsible space *zone_space* with the current *remain_bdd* (line 7). The resulting *zone_match* is then used to compute the LECs corresponding to individual records within the zonefile.

After processing a zonefile, AETHER updates *remain_bdd* by subtracting *zone_match* (line 8), ensuring that subsequent (less specific) zones operate only on the remaining query space. Once all zonefiles have been processed, the final *remain_bdd* characterizes the query space that matches no zonefile and thus constitutes the REFUSE LEC (line 10-11).

Zonefile-level match-action table computation. In Algorithm 2, given a zonefile and its allocated query space, its LECs are primarily divided based on the resource records it contains. First, we aggregate records with the same *rname* and *rtype*, as these records evidently correspond to the result of the same query (line 2).

AETHER calculates record priorities using the DNSRank function, which sorts records by type and wildcard status in decreasing order: *NS*, *DNAME*, non-wildcard *CNAME*, wildcard *CNAME*, and other records (line 3). This ensures query space is allocated sequentially to maintain mutually exclusive LECs and avoid conflicts. Special handling is required for *DNAME* and *CNAME* records as they exhibit dual behaviors; for instance, a *DNAME* record for *a.com* both rewrites subdomains (e.g., *ra.com*) and returns results for the node itself. Consequently, these records generate two rules assigned to different ranks. By incorporating these action-specific priorities into the rank computation, AETHER rapidly determines the behavior of each rule during space allocation, completing the LEC construction.

Algorithm 2: Construct the LECs for a zonefile

```

1 Function ZoneLECs(records, zone_space):
2   agg_records[]  $\leftarrow$  records aggregated by rname and
   rtype;
3   sorted_records[]  $\leftarrow$  agg_records sorted by DNSRank
   function;
4   remain_bdd  $\leftarrow$  zone_space;
5   foreach (rname, rtype, adata, rank) in sorted_records do
6     if rtype == DNAME then
7       sorted_records.add(rname, rtype, adata, 2);
8     else if rtype == CNAME then
9       sorted_records.add(rname, rtype, adata, rank - 1)
10  foreach (rname, rtype, adata, rank) in sorted_records do
11    if rank > 3 and rtype == NS then
12      record_space  $\leftarrow$  GetSpace(rname, all_types, 1);
13      action  $\leftarrow$  (Delegate, adata);
14    else if rank > 3 and rtype == DNAME then
15      record_space  $\leftarrow$  GetSpace(rname, all_types, 2);
16      action  $\leftarrow$  (RewriteD, adata);
17    else if rank = 1 and rtype == CNAME then
18      record_space  $\leftarrow$  GetSpace(rname, all_types -
   CNAME, 0);
19      action  $\leftarrow$  (RewriteC, adata);
20    else
21      record_space  $\leftarrow$  GetSpace(rname, rtype, 0);
22      action  $\leftarrow$  (Answer, adata);
23    record_match  $\leftarrow$  record_space  $\cap$  remain_bdd;
24    remain_bdd  $\leftarrow$  remain_bdd - record_match;
25    LECtable  $\leftarrow$  LECtable  $\cup$  (record_match, action);
26  nx_action  $\leftarrow$  (NonExist, -);
27  LECtable  $\leftarrow$  LECtable  $\cup$  (remain_bdd, nx_action);
28  return LECtable;

```

3.3 Query Resolution Graph Construction

AETHER models the DNS resolution process using a *query resolution graph*, which is a directed graph spanning all authoritative nameservers.

Nodes. Each nameserver has one logical input port and multiple logical output ports. The input port receives incoming queries, while each output port delegates queries to other nodes or triggers a specific resolution action.

Edges. The graph contains two types of directed edges. Internal edges connect a nameserver's input port to its output ports, capturing the match-action logic within the nameserver. Each internal edge is annotated with an LEC, representing the subset of the query space that triggers the corresponding action. External edges connect output ports to the input ports of next-hop nameserver(s) implied by delegation actions, modeling inter-nameserver query forwarding.

Query Resolution Graph Construction. The construction of the query resolution graph begins with local nameserver modeling. AETHER represents the input port and output ports

Algorithm 3: AETHER symbolic execution logic

```

1 Function Resolve(s, q, k):
2   visit  $\leftarrow \bigcup pq$ , where  $pq$  represents all queries that have
   previously accessed s;
3   if  $q_{in} \cap visit \neq \emptyset$  then
4     Save relevant match information;
5     return;
6   foreach (match, action) in LECtable do
7      $q_{out} \leftarrow q_{in} \cap bdd$ ;
8     if  $q_{out} \neq \emptyset$  then
9       Save relevant match information in log;
10      if action.atype == Delegate then
11         $s_{out} \leftarrow$  nameserver authoritative for domain
        action.adata;
12        Resolve( $s_{out}$ ,  $q_{out}$ , k-1);
13      if action.atype  $\in \{RewriteC, RewriteD\}$  then
14         $q_{out} \leftarrow$  new query after rewrite;
15        Resolve(s,  $q_{out}$ , k-1);

```

of each match-action component as nodes, connecting them with internal edges annotated by these LECs. Finally, the global graph is established by iteratively linking each forward output port to the input port of the next-hop nameserver(s) implied by the delegation action, providing a complete symbolic map for full query-space verification.

4 SYMBOLIC QUERY EXECUTION

This section introduce symbolic execution on query resolution graph to cover all possible queries, achieving comprehensive coverage and accurate verification.

4.1 A Symbolic Query Algorithm

We now formally present the symbolic execution of query q in AETHER. As shown in Algorithm 3, the core function takes three parameters: the starting nameserver s , the symbolic query q , and a fuel parameter k (limits the maximum recursion depth to guarantee that the resolution process always terminates).

AETHER generates query traces by performing a DFS search for the query resolution graph. Starting from the input port of the initial authoritative nameserver with an input query q_{in} , the algorithm traverses the resolution graph. For each encountered action, the process follows two paths: 1) termination actions: If the action is a termination type, a final resolution log is directly generated, marking the end of the trace; 2) forwarding actions: If the action involves redirection, AETHER computes the forwarded query space q_{out} (which may be a subset of or identical to q_{in}), records the transition metadata for the log, and recursively forwards q_{out} to the subsequent nameservers.

This recursive process continues until every path reaches a terminal action, resulting in a set of complete traces. Notably,

since a single query q_{out} can be forwarded to multiple destination nameservers, different traces may share overlapping logs. However, this structural overlap does not compromise the independence or validity of individual query traces.

Property Check. Upon completing symbolic execution, we obtain all execution traces, each representing the behavior of an EC q . These traces enable verification by analyzing whether q 's behavior adheres to predefined assertions. Specifically, each trace captures the sequence of query resolution steps, including delegation, rewriting, and response generation, which can be systematically compared against expected outcomes. Finally, we list in Table 1 a few typical error representations and how they are verified in our model.

4.2 Matching Optimization

Although BDD is highly efficient in logical operations, as the number of rules on a nameserver increases, matching a query against each rule remains a time-consuming task. Consider an extreme case: there are 10K A records in a nameserver, each taking the form $x.extreme.com.$, where $x \in \{1, \dots, 10,000\}$. AETHER will correspondingly compute 10K LECs for this nameserver. When it receives a query $q : \langle 2.example.com., A \rangle$, to process q , we need to perform 10,000 logical AND operations, each with a time complexity of $O(|G| \cdot |G|)$, where $|G|$ is the number of nodes in the BDD. Thus, the final time complexity would be $10000 \cdot O(|G| \cdot |G|)$. If the server needs to handle multiple queries, this could become a performance bottleneck for the system.

To address this, we need to perform targeted filtering before formal matching to identify LECs that may potentially affect q . Both Veriflow [27] and Flash [15] utilize a multi-dimensional prefix Trie to efficiently discover or filter rules. Borrowing from this concept, we observe that LECs generated by Section 3.2 can back-trace to a specific record, allowing us to filter LECs related to the query through $rname$. Specifically, we introduce a *prefix* field to the query q , which denotes the common prefix of the query space and is initially empty. This allows us to determine whether to perform the logical AND operation by assessing if the *prefix* matches $rname$ (i.e., the *prefix* is a subdomain of $r.rname$ or $rname$ is a subdomain of the *prefix*) before proceeding with the matching process.

Returning to this example, We assume that the query $q : \langle 1.extreme.com., A, 1.extreme.com. \rangle$ is derived from the query $\langle *.extreme.net., A \rangle$ through rewriting due to the record $\langle *.extreme.net., CNAME, 1.extreme.com. \rangle$, meaning it matches a *RewriteC* action. Here, we define that when a query matches a *Delegate* type action, the *prefix* is set to $rname$; when it matches a *RewriteC* or *RewriteD* type action, the *prefix* is set to $adata$; other types of actions do not alter the *prefix*. When q reaches $ns.extreme.com$ again, we perform 10,000 comparison operations, ultimately only executing the logical

AND operation with $r2$. This reduces our time complexity to $O(10000 \cdot n + |G| \cdot |G|)$, where n is the length of $rname$.

5 INCREMENTAL VERIFICATION

This section presents an efficient algorithm for fast incremental verification when configuration files are updated.

5.1 LEC Update

Addition and deletion at the record level. We first focus on adding or deleting resource records within existing zonefiles. The key idea is *priority-based preemptive update*. In Algorithm 2, we can deduce two properties for each match-action rule when *atype* is not *NonExist*: 1) The *match* of each rule is mutually independent; 2) Each rule implicitly contains a field *rank*, which influences the calculation of $match_r$, meaning that rules with higher ranks are computed first. For each rule with type *NonExist*, we assign a priority of its original priority minus 1.

In the following, we let r denote an updated record, and let $space_r$, rk_r , and $match_r$ denote the responsible space, rank, and matching predicate (i.e., LEC) of a record, respectively.

Step 1. Compute the Matching Space for the Updated Record. When inserting or deleting a resource record, if the record shares the same name and type as an existing entry in the zonefile, the LEC partitions remain unchanged and only the *adata* field is updated. For deletions, this condition holds if the record matches the name and type of others but possesses a different *adata* value.

Otherwise, we add or delete a record r from this zonefile, and Equation (1) provides the method for calculating the *match* field of the updated record. When deleting a record, since the record has been precomputed and exists within the zonefile, we can directly retrieve the matching field. For the case of adding a record, we will preempt the query space from NONEXIST LECs and other lower-rank rules' LECs.

$$match_r = \begin{cases} match_r, & \text{delete } r \\ space_r \wedge \left(\bigvee_{rk_r > rk_{r'}} match_{r'} \right), & \text{add } r \end{cases} \quad (1)$$

Step 2. Update of Affected Records. Since we performed space preemption on the rules with lower ranks while calculating LECs, we now need to update the LECs of those affected records that were subject to space preemption. Equation (2) provides the algorithm for this operation. After the deletion of record r , the space held by r will be released, and all records with lower ranks (r') will sequentially preempt the released query space. In other words, r' cannot preempt the space $match_{r''}$ that is intended to be preempted by a higher-priority record r'' . For the case of adding a rule, r'

simply removes the query space previously taken by r .

$$match_{r'} = \begin{cases} match_{r'} \vee (space_{r'} \wedge match_r \wedge \\ \neg(\bigvee_{rk_{r''} > rk_{r'}} space_{r''})), & \text{delete } r \\ space_{r'} \wedge \neg match_r, & \text{add } r \end{cases} \quad (2)$$

Optimization. As mentioned in Section 4.2, logical operations still incur a significant time overhead. Therefore, we will now introduce some tricky methods to reduce the amount of logical operations. We begin by analyzing the process of computing the disjunction of all $match_{r'}$ terms as shown in Equation (1). We can transform the entire process into a form similar to " $space_r \wedge match_{r'_1} \wedge match_{r'_2} \dots$ ". By doing so, after each conjunction operation, we can check if $match_r$ is equal to $space_r$ in order to decide whether to terminate early, thus reducing the computational complexity. For Equation (2), we can adopt a similar approach by tracking the space released or preempted by r (initially $match_r$), which we denote as s . After each update to $match_{r'}$, we update s accordingly. When s becomes empty, we can terminate the traversal of r' . As for the disjunction of r'' , we can apply dynamic programming techniques to avoid recalculating the entire disjunction each time r' is updated. Additionally, we can accelerate the computation by comparing the $rname$ fields between rules to filter out unrelated or unaffected rules.

Addition and deletion at the zonefile level. To extend the aforementioned methodology to the addition or deletion of zonefiles on a nameserver ns , we discuss zonefile-level operations. In Algorithm 2, zonefiles on ns are sorted by the hierarchical depth of their origins, where the origin's hierarchy determines its rank. For instance, a top-level domain (e.g., com) is assigned a rank=1, while a second-level domain (e.g., went.com) is assigned rank=2. For a zonefile z whose bdd is updated, this can be interpreted as either implicitly adding a rule with $rank = 512$ (where the rule's bdd corresponds to the preempted query space from z) or implicitly deleting a rule with $rank = 0$ (where the rule's bdd corresponds to the newly acquired query space for z). Next, just update all the rules in z in the same way you would add or delete a rule.

5.2 Incremental Execution

To enhance verification efficiency while circumventing the computational burden of full symbolic execution, we propose a delta-update mechanism that selectively regenerates execution traces impacted by modifications to LECs.

Phase 1: Affected trace identification. When a zone file z is modified, the system analyzes dependencies via log relationships. It retrieves historical logs L linked to z and extracts the maximal preceding sub-trace T_{prev} , capturing nameserver states and query history. Input queries $Q = \{q_1, q_2, \dots, q_n\}$ are filtered using BDD-based checks to isolate relevant queries.

Phase 2: Partial symbolic re-execution. Recomputation begins at the nameserver hosting z , with execution parameters restored from T_{prev} . Input queries Q are replayed through the updated LEC configuration, pruning unchanged BDD subspaces. The system generates version-tagged logs T_{next} .

Phase 3: Trace integration and verification. The final synthesis aligns T_{prev} and T_{next} to reconstruct event timelines. Verification follows the approach outlined in Section 4.1.

6 EVALUATION

We implement a prototype of AETHER using Rust based on the OxiDD [19] framework and conduct extensive evaluations on AETHER. For all the zones in the dataset, the initial input query is $q : \langle *, * \rangle$, which means that for all possible domains and all the types of query, we want to know their response. Specifically, we study the following questions: (1) What is the capability of AETHER in verifying DNS configuration files? (§6.1) (2) What is the performance of AETHER in a real-world DNS dataset? (§6.2)

Experiment Environment. To maintain the uniformity of the experimental environment, the verification experiments all run on the platform configured with Ubuntu 20.04.3 LTS (kernel version 5.13.0-41-generic), equipped with two Intel Xeon Gold 6126 CPU (2.60GHz) and 128GB of DDR4 memory.

6.1 Capability Evaluation

To evaluate the capability of AETHER, we set up the experiment on the university dataset and evaluate the property that AETHER support.

Dataset. We collect the DNS information from a university with zonefiles from multiple nameservers to show the capability of AETHER. The university dataset includes multiple levels of subdomains (more than 2), and has 9,700 zonefiles with 134,349 resource records in total.

Metric. We record the number of DNS configuration errors that AETHER detects in this dataset.

Experiment result. The results in Table 3 demonstrate that AETHER successfully verified the normal DNS configuration errors in this dataset. Additionally, AETHER detects warnings that may trigger potential attacks (e.g., repeated rewriting operations leading to unconstrained behavior [30]). Since our logs contain the log information (e.g., *originalqueryvalues*, *querytypes*, *resultvalues*, and *locationinformation*). We can notify users of the number of rewrites involved in the execution path by counting the number of rewrites matched in each trace record. This helps users guard against potential attack risks. Additionally, we record matching information during the symbolic execution in algorithm3, including the number of hops for each trace path. For infinite loop behaviors that could potentially attack the parser (though our model does not include a parser, excessively high hop counts may lead to parser attacks), this mechanism provides early

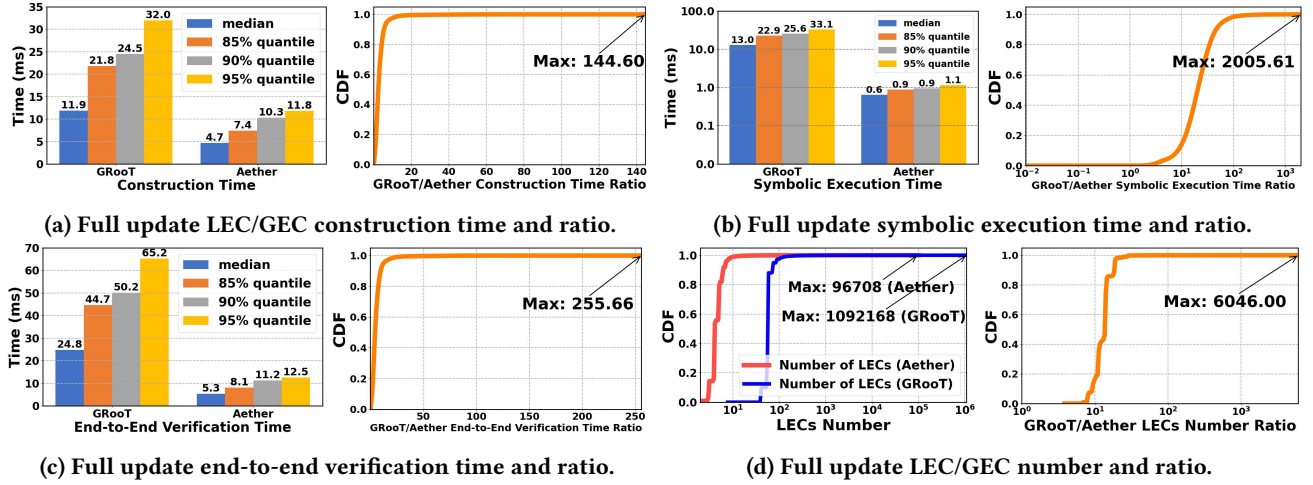


Figure 4: Full update experimental result using the Census dataset.

warning capabilities. This demonstrates that AETHER not only identifies common DNS configuration errors but also supports defenses against some DNS attacks.

Error Type	Error Cases
Delegation Inconsistency	5
Lame Delegation	7
Missing Glue Records	5
Cyclic Zone Dependency	8
Answer Inconsistency	0
Rewrite Loop	7
Rewrite Blackholing	2
Number of rewrites ≥ 2	15
Number of hops ≥ 5	22

Table 3: Errors detected on the university dataset and the number of cases AETHER reported. Cases in red are errors while cases in orange are warnings.

6.2 Performance Evaluation

To evaluate the performance of AETHER, we set up the experiment on the open-sourced dataset and evaluate the end-to-end verification time in the scenario of burst update and incremental update.

Dataset. We get the dataset from the real world named Census [7]. The census consists of 1,368,523 zones and over 65 million resource records. However, according to our test results with Bind9 [42], some files in this dataset violate the named-checkzone directive (e.g., quotes are not enclosed). These zones are defined as invalid zones, and we filter them out in our experiments (remaining 1,186,517 zones) because they are syntactically incorrect or incomplete. These files cannot be parsed successfully.

Metric. We evaluate the LEC construction time, symbolic execution time, and end-to-end verification time (LEC construction time + symbolic execution time) in burst and incremental update scenarios to represent the overall performance.

We also compare the number of LEC of AETHER with the number of EC of GROOT to show our contribution to the definition of LEC.

Experiment 1: Full update. We first evaluate AETHER in the scenario of full update (make the complete dataset available at once for full-volume evaluation).

LEC Construction. We compare the construction time of LECs and ECs, and the results are shown in Figure 4a. The median construction time for GROOT is 11.89 ms, while AETHER achieves a median of 4.65 ms. Additionally, at the 85%, 90%, and 95% percentiles, GROOT completes construction within 32.03 ms, compared to AETHER’s 11.83 ms. Furthermore, from the ratio figure the maximum speedup reaches 144.6 $\times$. This is because we support parallel computation during the generation of local equivalence classes, whereas GROOT’s architecture requires all zonfile equivalence classes to be computed sequentially.

Symbolic Execution. Figure 4b shows that the median execution time for GROOT is 12.96 ms, while AETHER achieves a median of just 0.64 ms. At the three quantiles, the maximum time for AETHER is 1.15 ms, compared to a minimum of 22.88 ms for GROOT. The ratio figure also indicates AETHER can achieve a maximum speedup of 2005.61 $\times$. Because AETHER generates fewer equivalence classes, it reduces the need to match repetitive behaviors during symbolic execution, thereby improving the efficiency of symbolic execution. By analyzing the structures of GROOT and AETHER, we observe that GROOT constructs an interpretation graph for each EC to perform symbolic execution, while AETHER matches corresponding rules for each LEC to execute symbolically. This reveals that the complexity of subsequent verification processes scales linearly with the number of equivalence classes. We record the number of ECs generated by AETHER on each zonefile and compare it with GROOT. We record the number

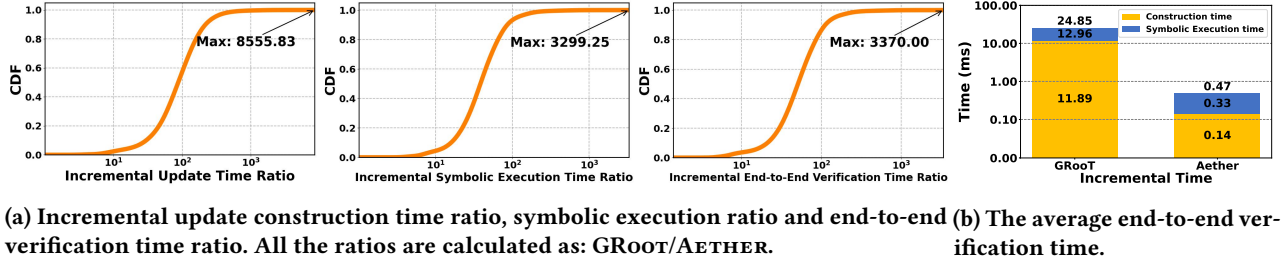


Figure 5: Incremental update experimental results using the Census dataset.

of ECs generated by AETHER on each zonefile and compare it with GROOT. The experimental results are shown in Figure 4d indicate that for all zones, the number of ECs generated by GROOT is greater than that produced by AETHER, with a maximum increase of 6046 $\times$. For AETHER, the maximum EC number is 96708, whereas for GROOT is 1092168. This demonstrates the reason that AETHER can perform better than GROOT in symbolic execution.

End-to-end verification. AETHER achieves a median runtime of 5.29 ms, compared to 44.67 ms for GROOT. Quantile analysis further highlights this improvement: the maximum runtime for GROOT is 65.15 ms, whereas AETHER reduces it to just 12.54 ms (Figure 4c). There is a maximum speedup of 255.66 $\times$ of end-to-end verification, as shown in the ratio figure. And as we have mentioned before, for 1,000,000 zones AETHER can finish verification in 140.56s that achieves 80.87 $\times$ speed up.

Experiment 2: Incremental update. We randomly delete and update records in census and then evaluate the performance of AETHER and GROOT. For each zone, we randomly select a zonefile to modify according to the following rules: first, we randomly choose a record type from the rtype. We suppose the number of records in the zonefile as num and calculate $Bound = \max(\min(num \times 0.01, 10), 1)$. Then, within the range of $[1, Bound]$, we generate a random number to create the corresponding number of records. Finally, we generate another random number within the range of $[1, Bound]$ to delete the specified number of records. Throughout the running period (Figure 5a), AETHER consistently outperforms GROOT, achieving a peak ratio of up to 8555.83 $\times$. In symbolic execution, AETHER demonstrates speedups across all zones, with a maximum speedup of 3299.25 $\times$. Similarly, in end-to-end verification, AETHER achieves speedups in all zones, reaching a maximum of 3370 $\times$. Figure 5b shows the mean of incremental end-to-end verification time component of AETHER and GROOT. Notably, AETHER demonstrates a speedup of 82.57 $\times$, while symbolic execution achieves a speedup of 39.27 $\times$. Because AETHER only needs to update the query space affected by the modified rule, while GROOT must re-collect the zone file, generate equivalence classes, and perform symbolic execution to support rule changes.

7 RELATED WORK

DNS Verification. DNS configuration verification tools [10, 23, 30, 48] model authoritative DNS resolution and explore feasible query paths to detect misconfigurations; the formal-method-based work [30] additionally studies potential resolver-side attacks. Octopus [48] proposes a distributed verification architecture and relies on string-based representations for query-space reasoning. VeriDNS [10] builds on Octopus and adds incremental verification, but its incremental checking is approximate and thus may miss affected behaviors. Complementary to configuration verification, Ironsides [6] and SCALE [24] focus on bugs in nameserver implementations, while RHINE [11] proposes new DNS authentication mechanisms. In contrast, AETHER targets DNS misconfigurations and achieves higher verification efficiency than prior configuration verifiers [23, 30, 48]; moreover, AETHER supports complete incremental verification, unlike VeriDNS [10].

Data-Plane Verification. Prior work on data-plane verification (DPV) [1, 17, 22, 26, 27, 31, 32, 41, 46, 47, 51, 53–55, 58, 59] reasons about forwarding correctness by partitioning the fixed-length packet header space (e.g., 5-tuples) into equivalence classes, and has developed a rich set of optimizations for scalability [15, 40, 50] and incremental verification [5, 17, 22, 25, 27, 59, 60]. In contrast to global EC construction, Batfish [4] and PPV [56] verify directly over locally aggregated spaces without computing globally refined ECs. AETHER follows this LEC-based reasoning paradigm, but generalizes it to authoritative DNS by supporting variable-length, hierarchical domain names and DNS-specific behaviors such as DNAME rewrites, enabling both scalable end-to-end verification and complete incremental verification.

8 CONCLUSION

We present AETHER, a novel DNS verification framework that achieves efficient and incremental verification through LEC-native query resolution graph. Experimental results demonstrate its effectiveness. In future work, we plan to investigate automatic methods for diagnosing and repairing erroneous DNS configurations.

Ethics. We adhered to standard responsible-disclosure practices for the identified error. The relevant parties have confirmed our findings and are assessing potential mitigations.

REFERENCES

- [1] Ehab Al-Shaer and Saeed Al-Haj. 2010. FlowChecker: Configuration Analysis and Verification of Federated OpenFlow Infrastructures. In *SafeConfig'10*. ACM, 37–44.
- [2] Oracle and/or its affiliates. 2020. dig - DNS lookup utility. https://docs.oracle.com/cd/E88353_01/html/E37839/dig-1.html. (2020). Accessed: January 16, 2025.
- [3] David Barr. 1912. Common DNS Operational and Configuration Errors. RFC 1912. (1912). <https://doi.org/10.17487/RFC1912>
- [4] Batfish. 2025. Batfish. <https://github.com/batfish/batfish>. (2025).
- [5] Ryan Beckett and Aarti Gupta. 2022. Katra: Realtime Verification for Multilayer Networks. In *NSDI'22*. USENIX, 617–634.
- [6] Martin Carlisle and Barry Fagin. 2012. IRONSIDES: DNS with no single-packet denial of service or remote code execution vulnerabilities. In *GLOBECOM'12*. IEEE, 839–844.
- [7] DNS Census2013. 2013. DNS Census2013. (2013). <https://dnscensus2013.neocities.org/> Accessed: January 16, 2025.
- [8] Cloudflare. 2023. 1.1.1.1 lookup failures on October 4, 2023. <https://blog.cloudflare.com/1-1-1-1-lookup-failures-on-october-4th-2023>. (2023). Accessed: January 16, 2025.
- [9] Internet Systems Consortium. 2009. Linux man page: named-checkzone. <https://linux.die.net/man/8/named-checkconf>. (2009). Accessed: January 16, 2025.
- [10] Haizhou Du, Kaiqiang Hu, Yao Wang, and Zhihao Guo. 2025. VeriDNS: Incremental Distributed Verification of DNS Configurations. *Computer Networks* (2025), 111929.
- [11] Huayi Duan, Rubén Fischer, Jie Lou, Si Liu, David Basin, and Adrian Perrig. 2023. {RHINE}: Robust and High-performance Internet Naming with {E2E} Authenticity. In *NSDI'23*. USENIX, 531–553.
- [12] Carlos Perez etc. 2014. dnsrecon. <https://github.com/darkoperator/dnsrecon>. (2014). Accessed: January 16, 2025.
- [13] Incident Report for npm. 2018. DNS misconfiguration cached in ISP DNS caches. <https://status.npmjs.org/incidents/v22ffls5cd6h>. (2018). Accessed: January 16, 2025.
- [14] James Fryman. 2014. DNS Outage Post Mortem. <https://github.blog/2014-01-18-dns-outage-post-mortem>. (2014). Accessed: January 16, 2025.
- [15] Dong Guo, Shenshen Chen, Kai Gao, Qiao Xiang, Ying Zhang, and Y. Richard Yang. 2022. Flash: fast, consistent data plane verification for large-scale network settings. In *SIGCOMM'22*. ACM, 314–335.
- [16] Paul E. Hoffman and Patrick McManus. 2018. DNS Queries over HTTPS (DoH). RFC 8484. (2018). <https://www.rfc-editor.org/rfc/rfc8484.html>
- [17] Alex Horn, Ali Kheradmand, and Mukul Prasad. 2017. Delta-Net: Real-Time Network Verification Using Atoms. In *NSDI'17*. USENIX, 735–749.
- [18] Check Host. 2020. Check Host. <http://check-host.net/check-dns>. (2020). Accessed: January 16, 2025.
- [19] Nils Husung, Clemens Dubsclaff, Holger Hermanns, and Maximilian A. Köhl. 2024. OxiDD: A Safe, Concurrent, Modular, and Performant Decision Diagram Framework in Rust. In *TACAS'24*. https://doi.org/10.1007/978-3-031-57256-2_13
- [20] InfinityFree. 2019. DNS Outage at iFastNet: Softaculous down. <https://forum.infinityfree.com/t/dns-outage-at-ifastnet-softaculous-down/19374>. (2019). Accessed: January 16, 2025.
- [21] Internet Systems Consortium, Inc. 2022. BIND - DNS Software. <https://www.isc.org/bind/>. (2022). Accessed: January 16, 2025.
- [22] Karthick Jayaraman, Nikolaj Bjørner, Jitu Padhye, Amar Agrawal, Ashish Bhargava, Paul-Andre C Bissonnette, Shane Foster, Andrew Helwer, Mark Kasten, Ivan Lee, et al. 2019. Validating Datacenters at Scale. In *SIGCOMM'19*. ACM, 200–213.
- [23] Siva Kesava Reddy Kakarla, Ryan Beckett, Behnaz Arzani, Todd Millstein, and George Varghese. 2020. GRooT: Proactive verification of dns configurations. In *SIGCOMM'20*. ACM, 310–328.
- [24] Siva Kesava Reddy Kakarla, Ryan Beckett, Todd Millstein, and George Varghese. 2022. {SCALE}: Automatically finding {RFC} compliance bugs in {DNS} nameservers. In *NSDI'22*. USENIX, 307–323.
- [25] Peyman Kazemian, Michael Chan, Hongyi Zeng, George Varghese, Nick McKeown, and Scott Whyte. 2013. Real Time Network Policy Checking Using Header Space Analysis. In *NSDI'13*. USENIX, 99–111.
- [26] Peyman Kazemian, Michael Chang, Hongyi Zeng, George Varghese, Nick McKeown, and Scott Whyte. 2013. Real time network policy checking using header space analysis. In *NSDI'13*. USENIX, 99–111.
- [27] Ahmed Khurshid, Wenxuan Zhou, Matthew Caesar, and P. Brighten Godfrey. 2012. Veriflow: verifying network-wide invariants in real time. 42, 4 (2012), 467–472.
- [28] J. Klensin. 2003. RFC3467: Role of the Domain Name System (DNS). (2003).
- [29] Si Liu, Huayi Duan, Lukas Heimes, Marco Bearzi, Jodok Vieli, David Basin, and Adrian Perrig. 2023. A Formal Framework for End-to-End DNS Resolution. In *SIGCOMM'23*. ACM, 932–949.
- [30] Si Liu, Huayi Duan, Lukas Heimes, Marco Bearzi, Jodok Vieli, David Basin, and Adrian Perrig. 2023. A Formal Framework for End-to-End DNS Resolution. In *SIGCOMM'23*. ACM, 932–949.
- [31] Nuno P Lopes, Nikolaj Bjørner, Patrice Godefroid, Karthick Jayaraman, and George Varghese. 2015. Checking Beliefs in Dynamic Networks. In *NSDI'15*. USENIX, 499–512.
- [32] Haohui Mai, Ahmed Khurshid, Rachit Agarwal, Matthew Caesar, P Brighten Godfrey, and Samuel Talmadge King. 2011. Debugging the Data Plane with Anteater. *SIGCOMM'11*, 290–301.
- [33] Microsoft. 2020. Description of the DNSLint utility. (2020). <https://learn.microsoft.com/en-us/previous-versions/troubleshoot/windows-server/description-dnslint-utility> Accessed: January 16, 2025.
- [34] P. Mockapetris. 1987. Domain names - concepts and facilities. RFC 1034. (1987). <https://www.rfc-editor.org/rfc/rfc1034.html>
- [35] P. Mockapetris. 1987. Domain names - implementation and specification. RFC 1035. (1987). <https://www.rfc-editor.org/rfc/rfc1035.html>
- [36] Paul Mockapetris and Kevin J Dunlap. 1988. Development of the domain name system. In *Symposium proceedings on Communications architectures and protocols*. 123–133.
- [37] Moritz Müller, Matthew Thomas, Duane Wessels, Wes Hardaker, Taejoong Chung, Willem Toorop, and Roland van Rijswijk-Deij. 2019. Roll, Roll, Roll your Root: A Comprehensive Analysis of the First Ever DNSSEC Root KSK Rollover. In *IMC'19*. ACM, 1–14.
- [38] Inc MXToolBox. 2004. MXToolbox. <https://mxtoolbox.com>. (2004). Accessed: January 16, 2025.
- [39] Jeffrey Pang, Aditya Akella, Anees Shaikh, Balachander Krishnamurthy, and Srinivasan Seshan. 2004. On the responsiveness of DNS-based network control. In *SIGCOMM'14*. ACM, Association for Computing Machinery. <https://doi.org/10.1145/1028788.1028792>
- [40] Peng Peng, Xun Sun, Zhengtao Shen, Feiyang Ding, Jiawei Chen, Lizhao You, Weirong Jiang, Yongping Tang, and Feng Luo. 2025. HeTu: High-Performance Centralized Parallel Data-Plane Verification for Hyper-Scale DCNs. In *ICNP'25*. IEEE, 1–11.
- [41] Gordon D Plotkin, Nikolaj Bjørner, Nuno P Lopes, Andrey Rybalchenko, and George Varghese. 2016. Scaling Network Verification Using Symmetry and Surgery. In *SIGPLAN'16*. ACM, 69–83.
- [42] Keerthi PS. 2020. BIND9 check zone file - How we do it. (Mar 2020). <https://bobcares.com/blog/bind9-check-zone-file/> Accessed: January 16, 2025.
- [43] Kyle Schomp, Tom Callahan, Michael Rabinovich, and Mark Allman. 2013. On measuring the client-side DNS infrastructure. In *IMC '13*. ACM. <https://doi.org/10.1145/2504730.2504734>

- [44] A. Shaikh, R. Tewari, and M. Agrawal. 2001. On the effectiveness of DNS-based server selection. In *INFOCOM'01*. IEEE. <https://doi.org/10.1109/INFCOM.2001.916678>
- [45] Liam Tung. 2019. Azure global outage: Our DNS update mangled domain records, says Microsoft. <https://www.zdnet.com/article/azure-global-outage-our-dns-update-mangled-domain-records-says-microsoft/>. (2019). Accessed: January 16, 2025.
- [46] Huazhe Wang, Chen Qian, Ye Yu, Hongkun Yang, and Simon S Lam. 2015. Practical Network-Wide Packet Behavior Identification by AP Classifier. In *CoNEXT'15*. ACM, 1–13.
- [47] Huazhe Wang, Chen Qian, Ye Yu, Hongkun Yang, and Simon S Lam. 2017. Practical Network-Wide Packet Behavior Identification by AP Classifier. *IEEE/ACM Transactions on Networking* 25, 5 (2017), 2886–2899.
- [48] Yao Wang, Kexin Yu, Ziyi Wang, Kaiqiang Hu, Haizhou Du, Qiao Xiang, Xing Fang, Geng Li, Ruiting Zhou, Linghe Kong, and Jiwei Shu. 2024. Rethinking DNS Configuration Verification with a Distributed Architecture. In *APNet '24*. ACM, 23–30.
- [49] Zack Whittaker. 2021. A DNS outage just took down a large chunk of the internet. <https://techcrunch.com/2021/07/22/a-dns-outage-just-took-down-a-good-chunk-of-the-internet/>. (2021). Accessed: January 16, 2025.
- [50] Qiao Xiang, Chenyang Huang, Ridi Wen, Yuxin Wang, Xiwen Fan, Zaoying Liu, Linghe Kong, Dennis Duan, Franck Le, and Wei Sun. 2023. Beyond a Centralized Verifier: Scaling Data Plane Checking via Distributed, On-Device Verification. In *SIGCOMM'23*. ACM, 152–166.
- [51] Geoffrey G Xie, Jibin Zhan, David A Maltz, Hui bit Zhang, Albert Greenberg, Gisli Hjalmytsson, and Jennifer Rexford. 2005. On Static Reachability Analysis of IP Networks. In *INFOCOM'05*. IEEE, 2170–2183.
- [52] Tianyin Xu and Yuanyuan Zhou. 2015. Systems approaches to tackling configuration errors: A survey. *Comput. Surveys* 47, 4 (2015), 1–41.
- [53] Hongkun Yang and Simon S Lam. 2013. Real-Time Verification of Network Properties Using Atomic Predicates. In *ICNP'13*. IEEE, 1–11.
- [54] Hongkun Yang and Simon S Lam. 2016. Real-Time Verification of Network Properties Using Atomic Predicates. *IEEE/ACM Transactions on Networking* 24, 2 (2016), 887–900.
- [55] Hongkun Yang and Simon S Lam. 2017. Scalable Verification of Networks with Packet Transformers Using Atomic Predicates. *IEEE/ACM Transactions on Networking* 25, 5 (2017), 2900–2915.
- [56] Lizhao You, Jiahua Zhang, Yili Jin, Hao Tang, and Xiao Li. 2022. Fast Configuration Change Impact Analysis for Network Overlay Data Center Networks. *IEEE/ACM Transactions on Networking* 30, 1 (2022), 423–436.
- [57] Bojan Zdrnja, Nevil Brownlee, and Duane Wessels. 2007. Passive monitoring of DNS anomalies. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 129–139.
- [58] Hongyi Zeng, Shidong Zhang, Fei Ye, Vimalkumar Jeyakumar, Mickey Ju, Junda Liu, Nick McKeown, and Amin Vahdat. 2014. Libra: Divide and Conquer To Verify Forwarding Tables in Huge Networks. In *NSDI'14*. USENIX, 87–99.
- [59] Peng Zhang, Aaron Gember-Jacobson, Yueshang Zuo, Yuhao Huang, Xu Liu, and Hao Li. 2022. Differential Network Analysis. In *NSDI'22*. USENIX, 601–615.
- [60] Peng Zhang, Xu Liu, Hongkun Yang, Ning Kang, Zhengchang Gu, and Hao Li. 2020. APKeep: Realtime Verification for Real Networks. In *NSDI'20*. USENIX, 241–255.